MVC
+ Arquitectura Hexagonal

Arquitectura Modelo-Vista-Controlador
y Arquitectura de Puertos y Adaptadores

Explicadas con ejemplos en Java: paquetes mvc y hexagonal1

Full Stack – NF1 · Traducción al español (nombres de código en su idioma original)

Arquitectura MVC

Modelo-Vista-Controlador explicada con un ejemplo en Java: el paquete mvc

01 ¿Qué es MVC?

- Patrón de arquitectura que separa la aplicación en tres responsabilidades
- **Modelo**: datos y lógica de negocio
- **Vista**: presentación e interacción con el usuario
- **Controlador**: coordina el modelo y la vista
- **Objetivo**: bajo acoplamiento y alta cohesión
- Cada parte se puede cambiar, probar y reutilizar de forma independiente

02 Las tres piezas y su papel

MODELO	VISTA	CONTROLADOR
• Contiene las reglas de negocio • No sabe nada de la vista • No hace ningún System.out	• Muestra información • Lee la entrada del usuario • No toma decisiones	• Orquesta el flujo • Recibe datos de la vista • Llama al modelo y devuelve el resultado
Greeting **GreetingService**	**GreetingView**	**GreetingController**

03 Estructura del paquete

```
nf1.fullstack.arquictectures.mvc
|-- Main.java
|-- model
|   |-- Greeting.java
|   `-- GreetingService.java
|-- view
|   `-- GreetingView.java
`-- controller
    `-- GreetingController.java
```

- Un subpaquete por cada rol
- **Main** solo construye y conecta
- Dependencias: Controller → Model y Controller → View
- Model no depende de nadie
- View no conoce el Model

04 Modelo: datos y lógica de negocio

```
public class Greeting {
    private final String message;

    public Greeting(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }
}
```

```
public class GreetingService {
    public Greeting greet(String name) {
        DayOfWeek dow = LocalDate.now().getDayOfWeek();
        String salutation;
        if (dow == DayOfWeek.SATURDAY) {
            salutation = "Bon cap de setmana";
        } else if (dow == DayOfWeek.SUNDAY) {
            salutation = "Bon diumenge";
        } else { salutation = "Bon dia"; }
        String msg = String.format("%s, %s!", salutation, name);
        return new Greeting(msg);
    }
}
```

- **Greeting**: entidad inmutable que guarda el mensaje
- **GreetingService**: decide el saludo según el día de la semana (regla de negocio)
- El modelo no imprime ni lee nada: no depende de la consola

05 Vista: presentación

```
public class GreetingView {
    private final Scanner scanner = new Scanner(System.in);

    public String readName() {
        System.out.print("Introdueix el teu nom (o 'quit' per sortir): ");
        return scanner.nextLine().trim();
    }

    public void showGreeting(String greeting) {
        System.out.println(greeting);
    }

    public void showGoodbye() {
        System.out.println("Adéu!");
    }
}
```

- Única clase que toca System.in / System.out
- No tiene lógica: solo lee y muestra
- Se podría sustituir por una vista web o gráfica sin tocar el modelo

06 Controlador: coordinación

```
public class GreetingController {
    private final GreetingView view;
    private final GreetingService service;

    public GreetingController(GreetingView view, GreetingService service) {
        this.view = view; this.service = service;
    }

    public void run() {
        String name;
        while (!(name = view.readName()).equalsIgnoreCase("quit")) {
            Greeting greeting = service.greet(name);
            view.showGreeting(greeting.getMessage());
        }
        view.showGoodbye();
    }
}
```

- Recibe las dependencias por el constructor (inyección de dependencias)
- Bucle: leer nombre → pedir saludo al modelo → mostrarlo
- Decide cuándo terminar: la entrada 'quit'

07 Main: composición de la aplicación

```
public class Main {
    public static void main(String[] args) {
        GreetingView view = new GreetingView();
        GreetingService service = new GreetingService();
        GreetingController controller =
            new GreetingController(view, service);
        controller.run();
    }
}
```

- Crea las tres piezas y las conecta (composition root)
- Después cede el control a controller.run()
- Cambiar de vista o de servicio solo afecta a esta clase

08 Flujo de ejecución

De	A	Mensaje
Usuario	View	escribe 'Anna'
View	Controller	readName() → "Anna"
Controller	Model	greet("Anna")
Model	Controller	Greeting("Bon dia, Anna!")
Controller	View	showGreeting(message)
View	Usuario	Bon dia, Anna!

El bucle se repite hasta que el usuario escribe 'quit'; entonces showGoodbye() muestra «Adéu!».

09 Ventajas y buenas prácticas

- **Separación de responsabilidades**: cada clase tiene un único motivo para cambiar
- **Testabilidad**: GreetingService se prueba sin consola
- **Sustituible**: nueva vista (web, GUI) reutilizando modelo y controlador
- **Trabajo en equipo**: se puede desarrollar cada capa en paralelo

> **Cuidado:** no poner lógica de negocio en el controlador ni en la vista.
> **Cuidado:** el modelo no debe depender de la vista.

10 Resumen

Modelo	Vista	Controlador
• Datos y reglas	• Entrada y salida	• Coordina el flujo
`Greeting` `GreetingService`	`GreetingView`	`GreetingController`

Arquitectura Hexagonal

Puertos y adaptadores, explicada con el paquete hexagonal1

11 ¿Qué es la arquitectura hexagonal?

- Patrón también conocido como **puertos y adaptadores** (Alistair Cockburn)
- El **dominio** (lógica de negocio) es el centro de la aplicación
- El dominio define **puertos**: interfaces de lo que necesita
- Los **adaptadores** implementan los puertos (consola, fichero, web, BD...)
- Las dependencias apuntan siempre hacia el dominio
- **Objetivo**: aislar la lógica de negocio de la tecnología externa
- Se puede cambiar o añadir un adaptador sin tocar el dominio

12 Las tres piezas y su papel

DOMINIO	PUERTO	ADAPTADORES
• Contiene las reglas de negocio • No sabe nada de la consola ni del fichero • Solo conoce el puerto	• Interfaz que define qué necesita el dominio • Es un contrato • Ninguna implementación	• Implementan el puerto • Hablan con el mundo exterior • Intercambiables • Sin lógica de negocio
Greeting **GreetingService**	**OutputPort**	**ConsoleOutputAdapter** **FileOutputAdapter** **CompositeOutputAdapter**

13 Estructura del paquete

```
nf1.fullstack.arquictectures.hexagonal1
|-- Main.java
|-- domain
|   |-- Greeting.java
|   `-- GreetingService.java
|-- appplication
|   `-- OutputPort.java
`-- adapter
    |-- ConsoleOutputAdapter.java
    |-- FileOutputAdapter.java
    `-- CompositeOutputAdapter.java
```

- **domain**: el núcleo, sin dependencias técnicas
- **appplication**: contiene el puerto (OutputPort)
- **adapter**: implementaciones concretas del puerto
- **Main** solo construye y conecta
- Dependencias: GreetingService → OutputPort y Adapters → OutputPort
- El dominio no depende de los adaptadores
- Los adaptadores dependen del puerto (inversión)

14 Dominio: datos y lógica de negocio

```
public class Greeting {
    private final String message;

    public Greeting(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }
}
```

```
public class GreetingService {
    private final OutputPort output;

    public GreetingService(OutputPort output) {
        this.output = output;
    }

    public void greet(String name) {
        Greeting greeting = new Greeting("Hola, " + name + "!");
        output.send(greeting.getMessage());
    }
}
```

- **Greeting**: entidad inmutable que guarda el mensaje
- **GreetingService**: crea el saludo y lo envía a través del puerto
- El dominio recibe el puerto por el constructor y no sabe qué adaptador hay detrás

15 Puerto: el contrato del dominio

```
public interface OutputPort {
    void send(String message);
}
```

- Interfaz en el paquete appplication
- Define qué necesita el dominio: enviar un mensaje
- No dice cómo ni dónde se envía
- El dominio depende de esta abstracción, no de una clase concreta
- Un solo método: send(String)
- Tantos adaptadores como hagan falta

> **¿Por qué es importante?** Inversión de dependencias: el dominio define la interfaz y los adaptadores se adaptan a ella.

16 Adaptadores: implementan el puerto

```
public class ConsoleOutputAdapter
        implements OutputPort {
    public void send(String m) {
        System.out.println(m);
    }
}
```

```
public class FileOutputAdapter implements OutputPort {
    private static final String FILE_PATH = "greetings.txt";

    public void send(String message) {
        Files.write(Paths.get(FILE_PATH),
            (message + System.lineSeparator()).getBytes(),
            StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }
}
```

```
public class CompositeOutputAdapter
        implements OutputPort {
    private final List<OutputPort> adapters;

    public void send(String message) {
        for (OutputPort a : adapters)
            a.send(message);
    }
}
```

- **ConsoleOutputAdapter**: escribe por consola
- **FileOutputAdapter**: añade el mensaje a greetings.txt
- **CompositeOutputAdapter**: reenvía a una lista de adaptadores
- El dominio no puede distinguir un adaptador de otro

17 Main: composición de la aplicación

```
public class Main {
    public static void main(String[] args) {
        ConsoleOutputAdapter console = new ConsoleOutputAdapter();
        FileOutputAdapter file = new FileOutputAdapter();
        CompositeOutputAdapter composite =
            new CompositeOutputAdapter(List.of(console, file));
        GreetingService service = new GreetingService(composite);
        Scanner scanner = new Scanner(System.in);
        System.out.print("Escriu el teu nom: ");
        service.greet(scanner.nextLine());
    }
}
```

- Crea los adaptadores y el servicio de dominio
- Los conecta: composition root
- Inyecta el adaptador compuesto en el dominio
- Lee el nombre e invoca greet()
- Cambiar de adaptador solo afecta a Main

18 Flujo de ejecución

De	A	Mensaje
Usuario	Main	escribe 'Anna'
Main	GreetingService	greet("Anna")
GreetingService	Composite	send("Hola, Anna!")
Composite	Console / File	send(...) a cada adaptador
Console	Usuario	Hola, Anna! (consola)
File	greetings.txt	+ línea en greetings.txt

El dominio solo conoce OutputPort; el CompositeOutputAdapter envía el mensaje a consola y a fichero.

19 Hexagonal vs MVC

MVC	Hexagonal
• Separa presentación, control y modelo • Organiza por roles (capas) • El controlador conoce modelo y vista • Pensado para la interfaz de usuario	• Aísla el dominio del exterior • Organiza por puertos y adaptadores • El dominio solo conoce sus puertos • Pensado para entrada y salida de cualquier tipo

20 Ventajas y buenas prácticas

- Dominio independiente de la tecnología (consola, fichero, web, BD)
- **Testabilidad**: se prueba GreetingService con un puerto falso
- **Extensibilidad**: nuevos adaptadores sin modificar el dominio
- **Flexibilidad**: combinar salidas con el Composite

> **Cuidado:** el dominio no debe importar clases de los adaptadores.
> **Cuidado:** no poner lógica de negocio en los adaptadores.

21 Resumen

Dominio	Puerto	Adaptadores
• Datos y reglas	• Contrato de salida	• Implementan el puerto
Greeting **GreetingService**	**OutputPort**	**Console** **File** **Composite**

Nota: el código Java se mantiene tal cual en el original (nombres de clases, métodos, variables, paquetes y cadenas de texto), por lo que los mensajes del programa siguen en catalán.